📌 DOM in JavaScript

📌 What is the DOM?

The **Document Object Model (DOM)** is a programming interface for web documents. It represents the structure of a document as a **tree of objects**, where each **node** corresponds to a part of the document.

✅ The DOM allows scripts to dynamically **access and update**:
✔ **Content**
✔ **Structure**
✔ **Style**

📌 Basic Concepts

📌 Document

✔ The **root** of the DOM tree.
✔ Represents the **entire HTML or XML document**.

📌 Element

✔ Each **HTML element** is an object within the DOM.
✔ Examples: `<div>`, `<p>`, `<a>`, etc.

📌 Nodes

✔ **Everything in the DOM is a node**.
✔ The **document itself** is a node.
✔ **Elements and text inside elements** are also nodes.

📌 Accessing the DOM

🔹 Selecting Elements

Method	Description
`getElementById(id)`	Selects an element by its ID.
`getElementsByClassName(className)`	Selects elements by their class name.
`getElementsByTagName(tagName)`	Selects elements by their tag name.
`querySelector(selector)`	Selects the first element that matches a CSS selector.
`querySelectorAll(selector)`	Selects all elements that match a CSS selector.

💡 **Examples:**

js
CopyEdit
```
// By ID
let header = document.getElementById('header');

// By class name
let items = document.getElementsByClassName('item');

// By tag name
let paragraphs = document.getElementsByTagName('p');

// By CSS selector
let firstItem = document.querySelector('.item');
let allItems = document.querySelectorAll('.item');
```

🔹 Traversing the DOM

Property	Description
`parentNode`	Access the parent node of an element.

`childNodes`	Access the child nodes of an element (returns a NodeList).
`firstChild`	Access the first child node of an element.
`lastChild`	Access the last child node of an element.
`nextSibling`	Access the next sibling node.
`previousSibling`	Access the previous sibling node.

💡 **Example:**

js
CopyEdit
```
let item = document.getElementById('item1');
let parent = item.parentNode;
let children = parent.childNodes;
let firstChild = parent.firstChild;
let lastChild = parent.lastChild;
let nextSibling = item.nextSibling;
let previousSibling = item.previousSibling;
```

📌 Manipulating the DOM

🔹 Creating Elements

Method	Description
`document.createElement(tagName)`	Creates a new element.

💡 **Example:**

js
CopyEdit
```
let newDiv = document.createElement('div');
```

🔹 Inserting Elements

Method	Description
`appendChild(child)`	Adds a child node to the end.
`prepend(child)`	Adds a child node to the start.
`before(node)`	Adds a node before the selected node.
`after(node)`	Adds a node after the selected node.
`classList.add("className")`	Adds a class to an element.

💡 **Example:**

js
CopyEdit
```
let parent = document.getElementById('container');
let newDiv = document.createElement('div');

// Both do the same thing
parent.appendChild(newDiv);
parent.append(newDiv);
```

🔹 Removing Elements

Method	Description
`removeChild(child)`	Removes a child node.
`node.remove()`	Removes the node itself.

💡 **Example:**

js
CopyEdit
```
let parent = document.getElementById('container');
```

```
let child = document.getElementById('item1');

// Removes the selected child
parent.removeChild(child);

// Removes the parent node itself
parent.remove();
```

🔹 Replacing Elements

Method	Description
`replaceChild(newChild, oldChild)`	Replaces an existing child node.

💡 **Example:**

js
CopyEdit
```
let parent = document.getElementById('container');
let newChild = document.createElement('div');
let oldChild = document.getElementById('item1');
parent.replaceChild(newChild, oldChild);
```

🔹 Cloning Elements

Method	Description
`cloneNode(deep)`	Clones a node.

💡 **Example:**

js
CopyEdit
```
let original = document.getElementById('item1');
let clone = original.cloneNode(true);
document.body.appendChild(clone);
```

📌 Modifying Element Attributes and Content

🔹 Attributes

Method	Description
`getAttribute(attrName)`	Gets the value of an attribute.
`setAttribute(attrName, value)`	Sets the value of an attribute.
`removeAttribute(attrName)`	Removes an attribute.

💡 **Example:**

js
CopyEdit
```
let link = document.getElementById('myLink');
let href = link.getAttribute('href'); // Get attribute
link.setAttribute('href', 'https://new-url.com'); // Set attribute
link.removeAttribute('href'); // Remove attribute
```

🔹 Inner HTML

Method	Description
`innerHTML`	Sets or gets the HTML content of an element.

💡 **Example:**

js
CopyEdit
```
let div = document.getElementById('content');
div.innerHTML = '<p>New content</p>'; // Set HTML content
let htmlContent = div.innerHTML; // Get HTML content
```

🔹 Text Content

Method	Description
`textContent`	Sets or gets the text content of an element.

💡 **Example:**

js
CopyEdit
```
let div = document.getElementById('content');
div.textContent = 'New text content'; // Set text content
let textContent = div.textContent; // Get text content
```

📌 Common DOM Methods & Properties

Method	Description
`document.createElement(tagName)`	Creates an element node.
`element.appendChild(child)`	Appends a child node.
`element.removeChild(child)`	Removes a child node.
`element.replaceChild(newChild, oldChild)`	Replaces a child node.
`element.cloneNode(deep)`	Clones an element.
`element.getAttribute(attrName)`	Gets an attribute.
`element.setAttribute(attrName, value)`	Sets an attribute.
`element.removeAttribute(attrName)`	Removes an attribute.
`element.innerHTML`	Gets or sets the HTML content.

`element.textContent`	Gets or sets the text content.

📌 EVENTS IN JAVASCRIPT

📌 What are Events?

Events are **actions or occurrences** that happen in the browser and can be **detected and handled by JavaScript**.

✅ **Examples of events:**
✔ **User interactions** (clicks, key presses, mouse movements)
✔ **Browser events** (page load, resize, scroll)

📌 Event Types

🔹 Mouse Events

Event	Description
`click`	Triggered when an element is clicked.
`dblclick`	Triggered when an element is double-clicked.
`mouseover`	Triggered when the mouse moves onto an element.
`mouseout`	Triggered when the mouse moves out of an element.
`mousemove`	Triggered when the mouse moves within an element.
`mousedown`	Triggered when a mouse button is pressed.
`mouseup`	Triggered when a mouse button is released.

🔹 Keyboard Events

Event	Description
`keydown`	Triggered when a key is pressed down.
`keyup`	Triggered when a key is released.
`keypress`	Triggered when a key is pressed and released (deprecated).

🔹 Form Events

Event	Description
`submit`	Triggered when a form is submitted.
`change`	Triggered when an input value changes.
`input`	Triggered when an input element is modified.
`focus`	Triggered when an element gains focus.
`blur`	Triggered when an element loses focus.

🔹 Window Events

Event	Description
`load`	Triggered when the entire page loads.
`resize`	Triggered when the window is resized.
`scroll`	Triggered when the window is scrolled.
`unload`	Triggered when the user leaves the page.

🔹 Document Events

Event	Description
`DOMContentLoaded`	Triggered when the initial HTML document is completely loaded and parsed.

📌 Event Handling in JavaScript

There are **three main ways** to handle events:
1️⃣ **Inline Event Handling**
2️⃣ **Direct Event Handling**
3️⃣ **Using Event Listeners (Recommended)**

🔹 1. Inline Event Handling (Not Recommended)

- JavaScript code is written **directly inside the HTML tag**.
- **Drawbacks:**
  ❌ Makes HTML bulky.
  ❌ Hard to maintain and debug.
  ❌ Has **lowest precedence** (easily overridden).

💡 **Example:**

```
<button onclick="alert('Button Clicked!')">Click Me</button>
```

🔹 2. Direct Event Handling (Limited Use Cases)

- Assigning event handlers **directly in JavaScript**.
- **Drawbacks:**
  ❌ Cannot attach multiple event handlers to the same element.

💡 **Syntax:**

```
element.event = function;
```

💡 **Example:**

```
let button = document.getElementById('myButton');
```

```
button.onclick = () => {
  alert("Button Clicked!");
};
```

🔹 3. Event Listeners (Best Practice)

- **Allows multiple event handlers on the same element.**
- **More flexible and scalable.**
- **Recommended for modern JavaScript development.**

💡 **Syntax:**

```
element.addEventListener(event, handler, [options]);
```

💡 **Example:**

```
let button = document.getElementById('myButton');

button.addEventListener('click', function() {
  alert("Button Clicked!");
});
```

🔹 Removing Event Listeners

Method	Description
`removeEventListener(event, handler, [options])`	Removes an attached event listener.

💡 **Example:**

```
let handler = function() {
  alert("Button Clicked!");
};

let button = document.getElementById('myButton');
button.addEventListener('click', handler);
button.removeEventListener('click', handler);
```

📌 Event Object

Every event listener **automatically receives an event object** containing **useful information** about the event.

🔹 Properties & Methods of Event Object

Property/Method	Description
`type`	The type of event (e.g., click, keydown).
`target`	The element that triggered the event.
`currentTarget`	The element to which the event handler is attached.
`preventDefault()`	Cancels the default action (e.g., stops a form submission).
`stopPropagation()`	Prevents further event propagation.

💡 **Example:**

```
let button = document.getElementById('myButton');

button.addEventListener('click', function(event) {
  console.log("Event Type:", event.type);
  console.log("Clicked Element ID:", event.target.id);
});
```

📌 Event Propagation

Events **flow through the DOM in two phases**:
1️⃣ **Bubbling Phase** (default) – Event starts at the target element and **bubbles up** to the root.
2️⃣ **Capturing Phase** – Event starts at the root and **goes down** to the target.

💡 **Example of Bubbling:**

```
let parent = document.getElementById('parent');
let child = document.getElementById('child');

parent.addEventListener('click', function() {
```

```
  console.log("Parent Clicked");
});

child.addEventListener('click', function() {
  console.log("Child Clicked");
});

// Clicking on child logs "Child Clicked" first, then "Parent Clicked".
```

💡 **Example of Capturing (Use `true` as third argument):**

```
parent.addEventListener('click', function() {
  console.log("Parent Clicked");
}, true);

child.addEventListener('click', function() {
  console.log("Child Clicked");
}, true);

// Clicking on child logs "Parent Clicked" first, then "Child Clicked".
```

📌 Event Delegation (Efficient Event Handling)

✅ **Instead of adding event listeners to multiple child elements, use ONE listener on the parent!**

💡 **Example:**

```
let list = document.getElementById('myList');

list.addEventListener('click', function(event) {
  if (event.target.tagName === 'LI') {
    alert("List Item Clicked: " + event.target.textContent);
  }
});
```

📌 Preventing Default Actions

Use `event.preventDefault()` to **stop default browser behavior**.

💡 **Example:**

```
let link = document.getElementById('myLink');

link.addEventListener('click', function(event) {
  event.preventDefault();
  alert("Link Click Prevented!");
});
```

📌 Stopping Event Propagation

Use `event.stopPropagation()` to **prevent event from propagating to parent elements**.

💡 **Example:**

```
let parent = document.getElementById('parent');
let child = document.getElementById('child');

parent.addEventListener('click', function() {
  console.log("Parent Clicked");
});

child.addEventListener('click', function(event) {
  event.stopPropagation(); // Stops bubbling
  console.log("Child Clicked");
});
```

📌 Common Use Cases of Event Handling

🔹 Form Validation

```
let form = document.getElementById('myForm');

form.addEventListener('submit', function(event) {
  let input = document.getElementById('myInput');
  if (input.value === '') {
    alert("Input cannot be empty!");
    event.preventDefault();
```

```
  }
});
```

🔹 Toggle Visibility

```
let toggleButton = document.getElementById('toggleButton');
let box = document.getElementById('box');

toggleButton.addEventListener('click', function() {
  box.style.display = (box.style.display === 'none') ? 'block' : 'none';
});
```

📌 EVENTS IN JAVASCRIPT

📌 What are Events?

Events are **actions or occurrences** that happen in the browser and can be **detected and handled by JavaScript**.

✅ **Examples of events:**
✔ **User interactions** (clicks, key presses, mouse movements)
✔ **Browser events** (page load, resize, scroll)

📌 Event Types

🔹 Mouse Events

Event	Description
`click`	Triggered when an element is clicked.
`dblclick`	Triggered when an element is double-clicked.
`mouseover`	Triggered when the mouse moves onto an element.
`mouseout`	Triggered when the mouse moves out of an element.
`mousemove`	Triggered when the mouse moves within an element.
`mousedown`	Triggered when a mouse button is pressed.
`mouseup`	Triggered when a mouse button is released.

🔹 Keyboard Events

Event	Description
`keydown`	Triggered when a key is pressed down.
`keyup`	Triggered when a key is released.
`keypress`	Triggered when a key is pressed and released (deprecated).

🔹 Form Events

Event	Description
`submit`	Triggered when a form is submitted.
`change`	Triggered when an input value changes.
`input`	Triggered when an input element is modified.
`focus`	Triggered when an element gains focus.
`blur`	Triggered when an element loses focus.

🔹 Window Events

Event	Description
`load`	Triggered when the entire page loads.
`resize`	Triggered when the window is resized.
`scroll`	Triggered when the window is scrolled.
`unload`	Triggered when the user leaves the page.

🔹 Document Events

Event	Description
`DOMContentLoaded`	Triggered when the initial HTML document is completely loaded and parsed.

📌 Event Handling in JavaScript

There are **three main ways** to handle events:
1️⃣ **Inline Event Handling**
2️⃣ **Direct Event Handling**
3️⃣ **Using Event Listeners (Recommended)**

🔹 1. Inline Event Handling (Not Recommended)

- JavaScript code is written **directly inside the HTML tag**.
- **Drawbacks:**
  ❌ Makes HTML bulky.
  ❌ Hard to maintain and debug.
  ❌ Has **lowest precedence** (easily overridden).

💡 **Example:**

```
<button onclick="alert('Button Clicked!')">Click Me</button>
```

🔹 2. Direct Event Handling (Limited Use Cases)

- Assigning event handlers **directly in JavaScript**.
- **Drawbacks:**
  ❌ Cannot attach multiple event handlers to the same element.

💡 **Syntax:**

```
element.event = function;
```

💡 **Example:**

```
let button = document.getElementById('myButton');
```

```
button.onclick = () => {
  alert("Button Clicked!");
};
```

🔹 3. Event Listeners (Best Practice)

- **Allows multiple event handlers on the same element.**
- **More flexible and scalable.**
- **Recommended for modern JavaScript development.**

💡 **Syntax:**

```
element.addEventListener(event, handler, [options]);
```

💡 **Example:**

```
let button = document.getElementById('myButton');

button.addEventListener('click', function() {
  alert("Button Clicked!");
});
```

🔹 Removing Event Listeners

Method	Description
`removeEventListener(event, handler, [options])`	Removes an attached event listener.

💡 **Example:**

```
let handler = function() {
  alert("Button Clicked!");
};

let button = document.getElementById('myButton');
button.addEventListener('click', handler);
button.removeEventListener('click', handler);
```

📌 Event Object

Every event listener **automatically receives an event object** containing **useful information** about the event.

🔹 Properties & Methods of Event Object

Property/Method	Description
`type`	The type of event (e.g., click, keydown).
`target`	The element that triggered the event.
`currentTarget`	The element to which the event handler is attached.
`preventDefault()`	Cancels the default action (e.g., stops a form submission).
`stopPropagation()`	Prevents further event propagation.

💡 **Example:**

```
let button = document.getElementById('myButton');

button.addEventListener('click', function(event) {
  console.log("Event Type:", event.type);
  console.log("Clicked Element ID:", event.target.id);
});
```

📌 Event Propagation

Events **flow through the DOM in two phases**:
1️⃣ **Bubbling Phase** (default) – Event starts at the target element and **bubbles up** to the root.
2️⃣ **Capturing Phase** – Event starts at the root and **goes down** to the target.

💡 **Example of Bubbling:**

```
let parent = document.getElementById('parent');
let child = document.getElementById('child');

parent.addEventListener('click', function() {
```

```
  console.log("Parent Clicked");
});

child.addEventListener('click', function() {
  console.log("Child Clicked");
});

// Clicking on child logs "Child Clicked" first, then "Parent Clicked".
```

💡 **Example of Capturing (Use `true` as third argument):**

```
parent.addEventListener('click', function() {
  console.log("Parent Clicked");
}, true);

child.addEventListener('click', function() {
  console.log("Child Clicked");
}, true);

// Clicking on child logs "Parent Clicked" first, then "Child Clicked".
```

📌 Event Delegation (Efficient Event Handling)

✅ **Instead of adding event listeners to multiple child elements, use ONE listener on the parent!**

💡 **Example:**

```
let list = document.getElementById('myList');

list.addEventListener('click', function(event) {
  if (event.target.tagName === 'LI') {
    alert("List Item Clicked: " + event.target.textContent);
  }
});
```

📌 Preventing Default Actions

Use `event.preventDefault()` to **stop default browser behavior**.

💡 **Example:**

```
let link = document.getElementById('myLink');

link.addEventListener('click', function(event) {
  event.preventDefault();
  alert("Link Click Prevented!");
});
```

📌 Stopping Event Propagation

Use `event.stopPropagation()` to **prevent event from propagating to parent elements**.

💡 **Example:**

```
let parent = document.getElementById('parent');
let child = document.getElementById('child');

parent.addEventListener('click', function() {
  console.log("Parent Clicked");
});

child.addEventListener('click', function(event) {
  event.stopPropagation(); // Stops bubbling
  console.log("Child Clicked");
});
```

📌 Common Use Cases of Event Handling

🔹 Form Validation

```
let form = document.getElementById('myForm');

form.addEventListener('submit', function(event) {
  let input = document.getElementById('myInput');
  if (input.value === '') {
    alert("Input cannot be empty!");
    event.preventDefault();
```

```
  }
});
```

🔹 Toggle Visibility

```
let toggleButton = document.getElementById('toggleButton');
let box = document.getElementById('box');

toggleButton.addEventListener('click', function() {
  box.style.display = (box.style.display === 'none') ? 'block' : 'none';
});
```

📌 PROMISES, CALLBACKS, AND ASYNC/AWAIT

📌 Callbacks

A **callback** is a function **passed as an argument** to another function. It is executed **after the completion of an asynchronous operation**.

🔹 Why Use Callbacks?

✔ Helps handle **asynchronous** tasks like **reading files, API calls, or fetching data**.
✔ **Executed only when the operation is done**.

🔹 Example: Callback Function

```
function fetchData(callback) {
  setTimeout(() => {
    console.log("Data fetched");
    callback("Fetched Data");
  }, 1000);
}

function handleData(data) {
  console.log(data);
}

fetchData(handleData);
```

✅ Here, `fetchData` takes a **callback function** (`handleData`).
✅ After 1 second, `"Data fetched"` is logged, then **calls `handleData`** with `"Fetched Data"`.

📌 Promises

A **Promise** is an object representing the **eventual completion or failure** of an asynchronous operation.

🔹 Promise States

State	Description
`Pending`	Initial state, neither fulfilled nor rejected.
`Fulfilled`	Operation completed successfully.
`Rejected`	Operation failed.

🔹 Creating a Promise

```
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true; // Simulate success or failure
      if (success) {
        resolve("Fetched Data");
      } else {
        reject("Error fetching data");
      }
    }, 1000);
  });
}
```

🔹 Handling Promises with `.then()` and `.catch()`

Method	Description
`then()`	Executes when the promise **resolves (success)**.
`catch()`	Executes when the promise **rejects (failure)**.

💡 **Example:**

```
fetchData()
```

```
  .then(data => {
    console.log(data); // Output: "Fetched Data"
  })
  .catch(error => {
    console.error(error);
  });
```

✅ If successful, `"Fetched Data"` is logged.
✅ If failed, the **error message** is logged.

🔹 Promise Chaining

Multiple promises can be **chained** together.

💡 **Example:**

```
function fetchNext(data) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log(`Data: ${data}`);
      resolve(`Next Data: ${data + 1}`);
    }, 1000);
  });
}

fetchNext(1)
  .then(res => fetchNext(2))
  .then(res => fetchNext(3))
  .then(res => console.log("All promises resolved"));
```

✅ **Each `.then()` waits for the previous promise to resolve** before executing.

📌 Async/Await (Modern Approach)

Async/Await is **syntactic sugar** over promises that makes **asynchronous code easier to read**.

🔹 Key Features

✔ **Cleaner, synchronous-looking syntax**.
✔ **Works with `try...catch` for error handling**.
✔ **Must be inside an `async` function**.

🔹 Using `async` and `await`

```
async function fetchData() {
  return new Promise(resolve => {
    setTimeout(() => resolve("Fetched Data"), 1000);
  });
}

async function handleData() {
  let data = await fetchData();
  console.log(data); // Output: "Fetched Data"
}

handleData();
```

✅ **`await` pauses execution** until the **promise resolves**.

🔹 Handling Multiple Async Calls

```
async function fetchMultiple() {
  let data1 = await fetchData();
  console.log("Data 1:", data1);

  let data2 = await fetchData();
  console.log("Data 2:", data2);
}

fetchMultiple();
```

✅ **Each `await` ensures** the previous operation **completes before continuing**.

🔹 Error Handling in Async/Await

Use `try...catch` to handle errors.

💡 **Example:**

```
async function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => reject("Error fetching data"), 1000);
  });
}

async function handleData() {
  try {
    let data = await fetchData();
    console.log(data);
  } catch (error) {
    console.error("Caught Error:", error);
  }
}

handleData();
```

✅ **Errors are caught using `try...catch` instead of `.catch()`.**

🔹 Immediately Invoked Async Function (IIFE)

To avoid manually calling a function, **wrap it in `()` and invoke immediately**.

💡 **Example:**

```
(async function() {
  let data = await fetchData();
  console.log(data);
})();
```

✅ **Runs automatically without needing to call it explicitly.**

📌 Summary of Promises vs Async/Await

Feature	Promises (`.then()/.catch()`)	Async/Await (`await`)
Readability	Less readable (callback nesting)	More readable (looks synchronous)
Error Handling	`.catch()` method	`try...catch`
Performance	Slightly better	Slightly slower due to `await`
When to Use	When chaining multiple async tasks	When writing clean, readable code

---

📌 API Calls in JavaScript

📌 What is an API?

An **API (Application Programming Interface)** allows applications to **communicate with each other**.

✅ **Common API Features:**
✔ **Retrieve data from a server**.
✔ **Send data to a server**.
✔ **Modify or delete data on a server**.

📌 HTTP Methods in APIs

Method	Description
`GET`	Retrieve data from the server.
`POST`	Send new data to the server.
`PUT`	Update existing data.
`DELETE`	Remove data from the server.

📌 Fetching Data from an API

Use the `fetch()` **function** to send API requests.

💡 **Example (Fetching Data):**

```
async function fetchData() {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts');
  let data = await response.json(); // Convert response to JSON
  console.log(data);
}

fetchData();
```

✅ `fetch()` is **asynchronous**, so we use `await` to pause execution until it resolves.

📌 Sending Data to an API (POST Request)

💡 **Example:**

```
async function sendData() {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: 'New Post', body: 'Post Content' })
  });

  let data = await response.json();
  console.log(data);
}

sendData();
```

✅ `JSON.stringify()` converts the object to JSON format.
✅ The `headers` define the **content type as JSON**.

📌 Error Handling in API Requests

Use `try...catch` to handle API errors.

💡 **Example:**

```
async function fetchData() {
  try {
    let response = await fetch('https://invalid-url.com');
    if (!response.ok) throw new Error(`HTTP Error! Status: ${response.status}`);
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("API Error:", error.message);
  }
}

fetchData();
```

✅ **`if (!response.ok)` checks for HTTP errors**.
✅ **Catches network errors and invalid responses**.

📌 Summary

✔ **Callbacks**: Used for simple async tasks but can lead to "callback hell".
✔ **Promises**: Provide better control over async operations with `.then()` and `.catch()`.
✔ **Async/Await**: Simplifies promises and makes code easier to read.
✔ **API Requests**: Use `fetch()` for making network requests.

📌 API (Application Programming Interface) in JavaScript

📌 What is an API?

An **API (Application Programming Interface)** is a **set of rules and protocols** that allows **different software systems to communicate** with each other.

✅ **Key Features of APIs:**
✔ Allows **data exchange** between a client and a server.
✔ Defines **endpoints** to access specific functionalities.
✔ Uses **HTTP methods (GET, POST, PUT, DELETE)** to interact with resources.

📌 Important API Concepts

Term	Definition
Base URL	The **main URL** for accessing an API (e.g., `https://api.example.com/`).
Endpoint	A **specific URL** within the base URL that provides access to a resource (e.g., `/users`).
AJAX (Asynchronous JavaScript and XML)	A method for making **asynchronous API calls** without refreshing the page.
JSON (JavaScript Object Notation)	The most commonly used **data format** for API responses.

📌 HTTP Methods in APIs

Method	Description
`GET`	Retrieve data from the server.
`POST`	Send new data to the server.
`PUT`	Update existing data on the server.
`DELETE`	Remove data from the server.

📌 Fetching Data from an API (`GET` Request)

The `fetch()` function is used to **retrieve data from an API**.

💡 **Example: Fetching Data from an API**

```
async function fetchData() {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts');
  let data = await response.json(); // Convert response to JSON
  console.log(data);
}

fetchData();
```

✅ `fetch()` **is asynchronous**; it returns a **Promise**.
✅ `await response.json()` **converts the response to JavaScript Object format**.

📌 Sending Data to an API (`POST` Request)

To send data to an API, we use the `POST` method.

💡 **Example: Sending Data to an API**

```
async function sendData() {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      title: 'New Post',
      body: 'Post Content'
    })
  });

  let data = await response.json();
  console.log(data);
}

sendData();
```

✅ `method: 'POST'` specifies the type of request.
✅ `headers` define the request format as **JSON**.
✅ `body` contains the data **converted into JSON format**.

📌 Updating Data in an API (PUT Request)

A `PUT` request updates existing data.

💡 **Example: Updating Data in an API**

```
async function updateData() {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      title: 'Updated Title',
      body: 'Updated Content'
    })
  });

  let data = await response.json();
  console.log(data);
}

updateData();
```

✅ **Similar to `POST`, but it updates existing data**.
✅ **Requires the ID of the resource being updated (`/posts/1`)**.

📌 Deleting Data in an API (DELETE Request)

A `DELETE` request removes a resource from the server.

💡 **Example: Deleting Data in an API**

```
async function deleteData() {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
    method: 'DELETE'
  });

  console.log("Data deleted successfully!");
}

deleteData();
```

✅ **Deletes the resource with ID 1**.
✅ **No need to send a request body**.

📌 Error Handling in API Requests

Always handle **network errors** and **server errors**.

💡 **Example: API Error Handling**

```
async function fetchData() {
  try {
    let response = await fetch('https://invalid-url.com');
```

```
    if (!response.ok) throw new Error(`HTTP Error! Status: ${response.status}`);

    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("API Error:", error.message);
  }
}

fetchData();
```

✅ **Uses `try...catch` to catch network errors**.
✅ **`if (!response.ok)` checks for HTTP errors**.

📌 API Response Status Codes

Status Code	Meaning
200 OK	Request was successful.
201 Created	Resource was successfully created.
400 Bad Request	Client sent an invalid request.

401 Unauthorized	Authentication is required.
404 Not Found	Resource was not found.
500 Internal Server Error	Server encountered an error.

📌 Authentication in APIs

APIs **often require authentication** to access protected data.

Method	Description
API Key	A secret key that must be sent with each request.
OAuth	A secure method for user authentication.
JWT (JSON Web Token)	A token-based authentication system.

💡 Example: Using an API Key in `fetch()`

```
async function fetchData() {
  let response = await fetch('https://api.example.com/data', {
    headers: {
      'Authorization': 'Bearer YOUR_API_KEY'
    }
```

```
  });

  let data = await response.json();
  console.log(data);
}

fetchData();
```

✅ **Sends an API Key inside the `Authorization` header**.

📌 Rate Limiting in APIs

To **prevent abuse**, APIs **limit the number of requests** a user can make.

Method	Description
Throttling	Limits the number of requests per second/minute.
Quota-based	Users have a daily/monthly request limit.
IP-based	Limits requests based on the user's IP address.

📌 API Best Practices

✔ **Always handle errors (`try...catch`).**
✔ **Use proper HTTP methods (`GET`, `POST`, `PUT`, `DELETE`).**
✔ **Check for API limits to avoid getting blocked.**
✔ **Secure API keys – never expose them in frontend code!**

📌 Practical Guide: How to Retrieve Data from an API

✅ **Step-by-Step Process**
1️⃣ Define an **async function**.
2️⃣ Call the `fetch()` function with the **API URL**.
3️⃣ Store the **response** in a variable.
4️⃣ Convert the response to **JSON** using `.json()`.
5️⃣ Access and display the **needed data**.

💡 **Example Code:**

```
async function fetchData() {

  // Step 2: Call the API

  const response = await fetch('https://jsonplaceholder.typicode.com/users');



  // Step 3: Convert response to JSON

  const data = await response.json();



  // Step 4: Access and display needed data

  data.forEach(user => {

    console.log(`User: ${user.name}, Email: ${user.email}`);

  });

}



// Step 5: Call the function
```

```
fetchData();
```

✅ **Fetches a list of users and prints their names and emails**.

📌 Summary

✔ **What is an API?** – A system that allows different applications to communicate.
✔ **HTTP Methods:** `GET`, `POST`, `PUT`, `DELETE`.
✔ **`fetch()` Method:** Used to retrieve/send data from/to an API.
✔ **Authentication:** Uses API keys, OAuth, or JWT.
✔ **Error Handling:** Always use `try...catch` and check `response.ok`.
✔ **Best Practices:** Secure API keys, handle errors, and respect rate limits.